EKSPLORACJA DANYCH · WYKŁAD 3

Pozyskiwanie i przetwarzanie danych

Paweł Gliwny
Wydział Fizyki i Informatyki Stosowanej, Uniwersytet Łódzki
2026/2027

Od źródła do modelu

Algorytm uczenia maszynowego potrzebuje **tabeli**: wiersze to obserwacje, kolumny to cechy.

źródło danych
↓ *skąd je wziąć?*
plik (CSV, JSON, Parquet…)
↓ *jak go poprawnie wczytać?*
ramka danych
↓ *co w niej jest nie tak?*
dane gotowe dla modelu

Plan wykładu

1. **Otwarte dane** — skąd brać zbiory, benchmarki.
2. **Formaty danych** — wczytywanie plików, polskie CSV, kodowanie.
3. **Przygotowanie danych** — najczęstsze problemy.
4. **Wartości odstające** — od prostych reguł do odległości.

> Brakującymi wartościami zajmiemy się na kolejnych zajęciach.

1

Otwarte dane

Źródła otwartych danych

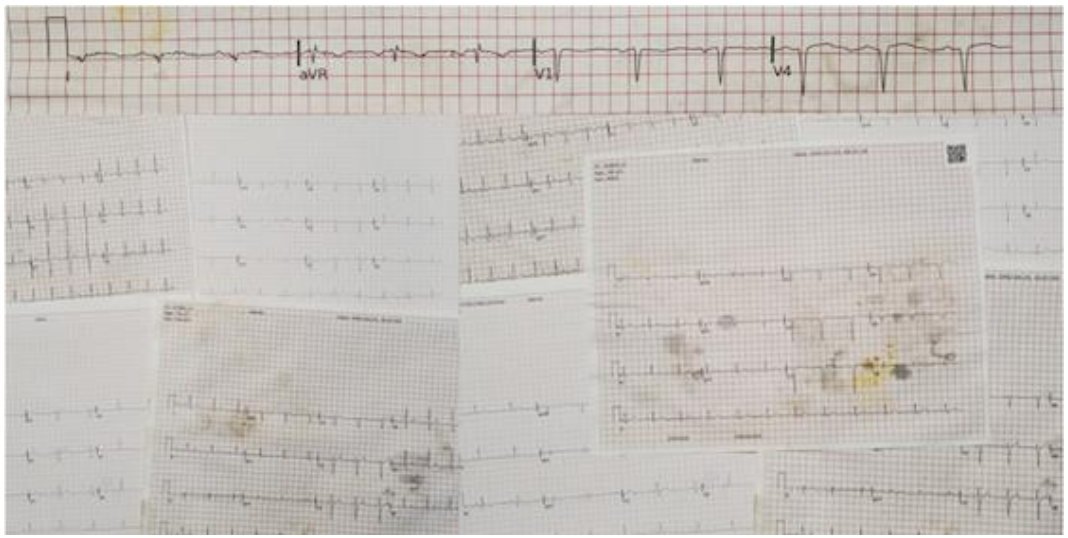
- **Kaggle** — kaggle.com
- **UCI Machine Learning Repository** — archive.ics.uci.edu
- **Hugging Face** — huggingface.co
- **Our World in Data** — ourworldindata.org
- **dane.gov.pl** — portal otwartych danych administracji
- portale otwartych danych miast

Kaggle

- Platforma konkursów *data science*.
- Dziesiątki tysięcy zbiorów danych z różnych dziedzin.
- Gotowe notatniki z analizami (zakładka *Code*, dawniej *Kernels*).
- Społeczność praktyków i naukowców.

Przykład konkursu Kaggle: digitalizacja EKG

- Zamiana danych **nieustrukturyzowanych** w **ustrukturyzowane**.
- *PhysioNet — Digitization of ECG Images*: odtworzenie szeregów czasowych EKG ze skanów i zdjęć papierowych wydruków.



Źródło: Kaggle, konkurs *PhysioNet — Digitization of ECG Images* link kaggle

UCI Machine Learning Repository

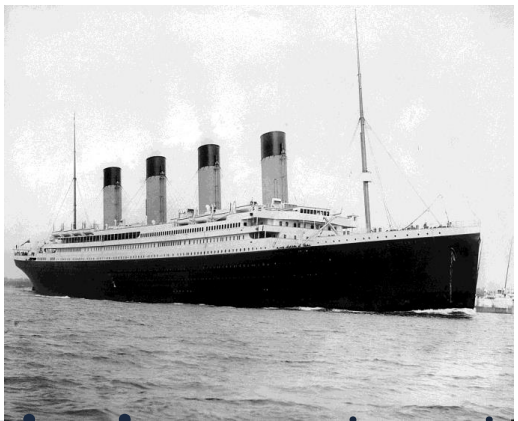
- Akademicka kolekcja zbiorów danych (University of California, Irvine).
- Klasyczne zbiory do nauki uczenia maszynowego.
- Dobrze udokumentowane.
- Standard w literaturze naukowej.

Popularne zbiory danych



Iris — kosaćce

E. Anderson, R. Fisher (1936)



Titanic — pasażerowie

katastrofa z 1912 r., popularny na Kaggle



Wine — klasyfikacja win

UCI ML Repository

Popularne zbiory — zalety i wady

Zalety

- małe — szybko się wczytują,
- zwykle dobrze przygotowane,
- dobrze udokumentowane,
- dostępne w bibliotekach (scikit-learn, seaborn),
- znane wyniki do porównań.

Wady

- wszyscy je znają — brak oryginalności,
- zbyt proste — nie oddają prawdziwych problemów,
- małe — nie sprawdzają skalowalności,
- brak współczesnych wyzwań (duże dane, czas rzeczywisty).

Kiedy ich używać? Nauka podstaw, testowanie algorytmów, prototypy, demonstracje na zajęciach.

Czy klasyczne zbiory są „czyste”?

Nie zawsze. Titanic (891 pasażerów) ma braki w trzech kolumnach:

Kolumna	Braki
`Cabin` — kabina	**77,1%**
`Age` — wiek	**19,9%**
`Embarked` — port	0,2%

Pytanie

Co zrobić z kolumną, w której brakuje ¾ wartości? A co z wiekiem? Wrócimy do tego przy brakujących wartościach.

Hugging Face

- Otwarta platforma dla AI: *Hub* modeli, zbiorów danych i aplikacji.
- **Setki tysięcy** zbiorów danych — głównie **tekst, obrazy i dźwięk**.
- To przede wszystkim zaplecze **uczenia głębokiego** (sieci neuronowe, modele językowe, widzenie komputerowe).
- Każdy zbiór ma opis (*dataset card*) i licencję.




```python
from datasets import load_dataset
recenzje = load_dataset("stanfordnlp/imdb")   # recenzje filmów + sentyment
```

Do klasycznego uczenia maszynowego na danych tabelarycznych częściej sięgamy po UCI, Kaggle czy portale otwartych danych.

Benchmarki

- **Benchmark** — wspólny „egzamin” do porównywania metod:

> **dane** + **zadanie** + **metryka** + **protokół** (jak liczymy wynik)

- Klasyczne przykłady:
  - **MNIST** — ręcznie pisane cyfry (klasyfikacja obrazów),
  - **ImageNet** — 1000 klas obiektów; przełom uczenia głębokiego w 2012 r.,
  - **UCI / OpenML** — zestawy zbiorów tabelarycznych do porównywania algorytmów.
- Dzięki benchmarkom wyniki z różnych artykułów da się porównać.

Benchmark analizy danych dla AI

EDA Benchmark (deepsense.ai, 2026):

- testuje **agentów AI** (modele językowe), które same mają przeprowadzić analizę danych,
- 10 syntetycznych zadań: czyszczenie danych, wykrywanie anomalii, szeregi czasowe — np. wadliwe stacje pomiaru powietrza we Wrocławiu,
- każdy model rozwiązuje zadanie 5 razy — liczy się też **powtarzalność**.

Pytanie

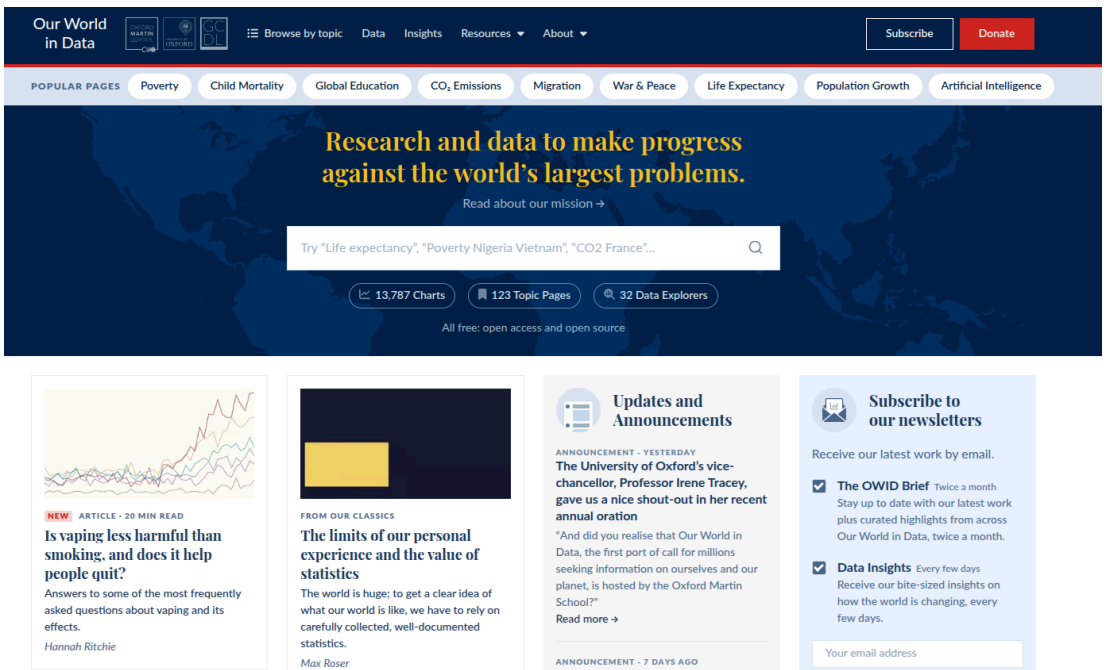
Czy najlepsze modele AI bezbłędnie analizują dane? Nie — i dlatego warto to umieć samemu.

Temat zaawansowany — wrócimy do niego pod koniec semestru.

Źródło: github.com/deepsense-ai/eda-benchmark · licencja CC BY-NC-ND 4.0

Our World in Data

- Dane o globalnych trendach i problemach.
- Demografia, ekonomia, środowisko, zdrowie.
- Dopracowane wizualizacje i analizy.
- Opisane źródła i metodologia.



Źródło: ourworldindata.org

dane.gov.pl

- Oficjalne dane polskiej administracji publicznej.
- Finanse publiczne, demografia, transport, środowisko.
- Otwarte formaty: CSV, JSON, XML; także API.
- Aktualizowane przez urzędy.

Uwaga
Pliki z polskich urzędów często mają **średnik** jako separator, **przecinek** dziesiętny i kodowanie **Windows-1250**.

Portale otwartych danych polskich miast

- Dane o transporcie, środowisku, budżecie, infrastrukturze.
- Formaty: CSV, JSON, XML, API — dostęp bezpłatny.
- Przykłady: Warszawa (api.um.warszawa.pl), Kraków (otwartedane.um.krakow.pl), Wrocław, Poznań.
- **Łódź:** strona otwarte miasto Łódź

Pytanie
Jakie dane o Łodzi chcielibyście analizować? Czy są dostępne?

2

Pliki tekstowe i CSV

Pliki tekstowe

- **CSV, TSV** — tabele: wiersz = linia, kolumny oddzielone separatorem.
- **TXT, LOG** — logi systemów, wyniki pomiarów.
- Bardzo częstszy format na portalach otwartych danych.

Zalety

- czytelne dla człowieka i każdego programu.

Wady

- duże na dysku, wolne w odczycie,
- **brak informacji o typach** i kodowaniu — trzeba je odgadnąć.

JSON, Excel, formaty binarne (Parquet) i bazy danych — na kolejnych zajęciach.

Krok 1: zanim wczytasz — obejrzyj plik

```
with open("dane.csv", encoding="utf-8") as f:
    for _ in range(5):
        print(f.readline().rstrip())
```

W Colabie także: `!head -n 5 dane.csv`

Szukamy odpowiedzi na pytania:

- jaki jest **separator**: , ; tabulator?
- czy jest **nagłówek**? czy nad tabelą są wiersze opisu?
- przecinek czy kropka **dziesiętna**?
- czy polskie znaki wyglądają poprawnie (**kodowanie**)?

Krok 2: `read_csv` — najważniejsze parametry

Parametr	Do czego?
`sep`	separator kolumn (`";"`, `"\t"`)
`decimal`, `thousands`	przecinek dziesiętny, spacja tysięcy
`encoding`	kodowanie (`"utf-8"`, `"cp1250"`)
`skiprows`	pominięcie wierszy opisu na początku
`comment`	pominięcie wierszy od `#`
`usecols`, `nrows`	tylko wybrane kolumny / wiersze
`na_values`	napisy oznaczające brak
`dtype`, `parse_dates`	typy kolumn, kolumny z datami

Plik z dwoma wierszami opisu nad tabelą:

```
df = pd.read_csv(
    "raport.csv",
    sep=";", decimal=",", thousands=" ",
    encoding="cp1250",
    skiprows=2,
    usecols=["rok", "wartość"],
    na_values=["-", "brak"],
)
```

Krok 3: sprawdź typy kolumn

```
# gminy.csv:
# teryt;gmina;data_pomiaru
# 0201011;Bolesławiec;2024-03-01

df = pd.read_csv("gminy.csv", sep=";")
df.dtypes       # teryt → int64 (201011!)
                # data_pomiaru → str

df = pd.read_csv("gminy.csv", sep=";",
                 dtype={"teryt": str},
                 parse_dates=["data_pomiaru"])
df.dtypes       # teryt → str ("0201011")
                # data_pomiaru → datetime64
```

- CSV **nie zapisuje typów** — pandas zgaduje je z treści.
- Kody (TERYT, PESEL, kody pocztowe) wyglądają jak liczby, ale nimi nie są — tracą **zera wiodące**.
- Daty trzeba wskazać ręcznie.

Po każdym wczytaniu: `df.dtypes`.

Krok 4: zapis i duże pliki

Zapis

```
df.to_csv("wynik.csv", index=False)

# dla polskiego Excela:
df.to_csv("wynik.csv", index=False,
          sep=";", decimal=",",
          encoding="utf-8-sig")
```

`utf-8-sig` dodaje znacznik BOM — Excel rozpozna wtedy polskie znaki.

Duże pliki

```
# podgląd bez wczytywania całości
pd.read_csv("duzy.csv.gz", nrows=5)

# przetwarzanie w kawałkach
suma = 0
for kawalek in pd.read_csv("duzy.csv.gz",
chunksize=100_000):
    suma += kawalek["x"].sum()
```

Pliki `.gz`, `.zip`, `.bz2` pandas rozpakowuje sam.

Ściągawka: wczytywanie CSV

1. **Obejrzyj** pierwsze linie surowego pliku.
2. Ustal separator, separator dziesiętny, kodowanie, wiersze do pominięcia.
3. Wczytaj: `pd.read_csv(...)` z właściwymi parametrami.
4. Sprawdź: `df.head()`, `df.shape`, `df.dtypes`.
5. Popraw typy: kody jako tekst (`dtype`), daty (`parse_dates`).

> Te same kroki wykonacie na laboratorium z plikiem z portalu Łodzi.

Zadanie: dane z portalu Łodzi

1. Wybierz zbiór na otwarte.miasto.lodz.pl i pobierz plik.
2. Wczytaj go (separator? kodowanie?).
3. Sprawdź format: **długi** czy **szeroki**?
4. Policz statystyki zbiorcze.
5. Zrób 2–3 wykresy i opisz jednym zdaniem, co widać.

Przykład: szkolnictwo wyższe w Łodzi 2012–2024

```
df = pd.read_csv("szkolnictwo_wysze.csv", sep=";")
df.head()
# name | rok | wartość | zmienna  ← format długi

szeroka = df.pivot(index="rok", columns="zmienna",
                   values="wartość")
szeroka.describe()
szeroka[["kobiety", "mężczyźni"]].plot(marker="o")
```

3

Przygotowanie danych do analizy

Przygotowanie danych

Przygotowanie danych zajmuje większość czasu analityka.

Główne zadania

- ładowanie danych,
- czyszczenie,
- przekształcanie (transformacja),
- reorganizacja.

W praktyce dane są...

- rozproszone w wielu źródłach, w różnych formatach i strukturach,
- nieprzygotowane do bezpośredniej analizy.

Najczęściej spotykane problemy w danych

Wartości odstające
Obserwacje leżące daleko od pozostałych.

Brakujące wartości
Pola, w których nie ma danych.

Dziś zajmiemy się **wartościami odstającymi** — na danych o pacjentach i na zbiorze Titanic.

4

Wartości odstające

Wartości odstające

- Obserwacje znajdujące się w dużej odległości od pozostałych punktów.
- Mogą być **błędem** (literówka, awaria czujnika) albo **prawdziwą**, choć rzadką obserwacją.
- W modelach mogą:
  - silnie wpływać na średnią, odchylenie standardowe i współczynniki regresji,
  - pogarszać dopasowanie dla typowych obserwacji.

> Nie usuwamy ich automatycznie — najpierw ustalamy, **skąd się wzięły**.

Skąd się biorą wartości odstające?

Przyczyna	Przykład	Co zrobić?
błąd wprowadzania	wiek 250 lat; wzrost „1,80” wśród wartości w cm	poprawić albo zamienić na NaN
kod zastępczy	`-999`, `9999` jako „brak danych”	zamienić na NaN (`na_values`)
inna jednostka	°F wśród °C, zł wśród tys. zł	przeliczyć
awaria czujnika	nagły skok lub długo stała wartość	oznaczyć, zamienić na NaN
rzadka, ale prawdziwa	bilet na Titanica za 512 £ (apartament)	zostawić, użyć metod odpornych
inna populacja	transakcja oszusta wśród zwykłych	to może być **cel** analizy!

Dane do ćwiczeń: pacjenci i udar

3000 pacjentów — **dane syntetyczne**, nie wyciągamy z nich wniosków medycznych: wiek, BMI, średni poziom glukozy, nadciśnienie, palenie, rodzaj pracy, udar (tak/nie).

	min	max
`age`	0,67	80
`bmi`	−5,2	105,6
`avg_glucose_level`	−21,4	1254,6

Pytanie

Które z tych wartości są w ogóle **możliwe**? Jak sprawdzić, czy 6-letnie dziecko może palić papierosy — w danych?

Źródło: EDA Benchmark, zadanie *identify-unusual-values* — github.com/deepsense-ai/eda-benchmark

Krok 1: wartości niemożliwe

```
pacjenci = pd.read_csv("synthetic_stroke_data.csv")

# progi z wiedzy dziedzinowej
zle_bmi = ~pacjenci["bmi"].between(10, 60)
zla_glukoza = ~(pacjenci["avg_glucose_level"]
                .between(20, 800))

zle_bmi.sum(), zla_glukoza.sum()   # (300, 300)

# to błędy, nie rzadkie obserwacje → NaN
pacjenci.loc[zle_bmi, "bmi"] = np.nan
pacjenci.loc[zla_glukoza,
             "avg_glucose_level"] = np.nan
```

- Ujemne BMI czy glukoza 1254 mg/dl to **szum techniczny**, nie pacjent.
- Nie potrzeba statystyki — wystarczy **wiedza dziedzinowa**.
- Łącznie 560 pacjentów ma co najmniej jedną niemożliwą wartość.

> Zawsze zaczynamy od tego kroku — inaczej błędy zniekształcą średnie i progi kolejnych metod.

Jedna obserwacja zmienia średnią

Miesięczne dochody 7 osób (tys. zł):

3 4 4 5 5 6 200

	bez 200	z 200
średnia	4,5	**32,4**
mediana	4,5	5,0

- Średnia i odchylenie standardowe są **nieodporne** na wartości odstające.
- **Statystyki odporne:** mediana, IQR, MAD (medianowe odchylenie bezwzględne).

Pytanie
Która liczba lepiej opisuje „typowy” dochód w tej grupie?

Krok 2: wykrywanie w jednej zmiennej

Metody graficzne

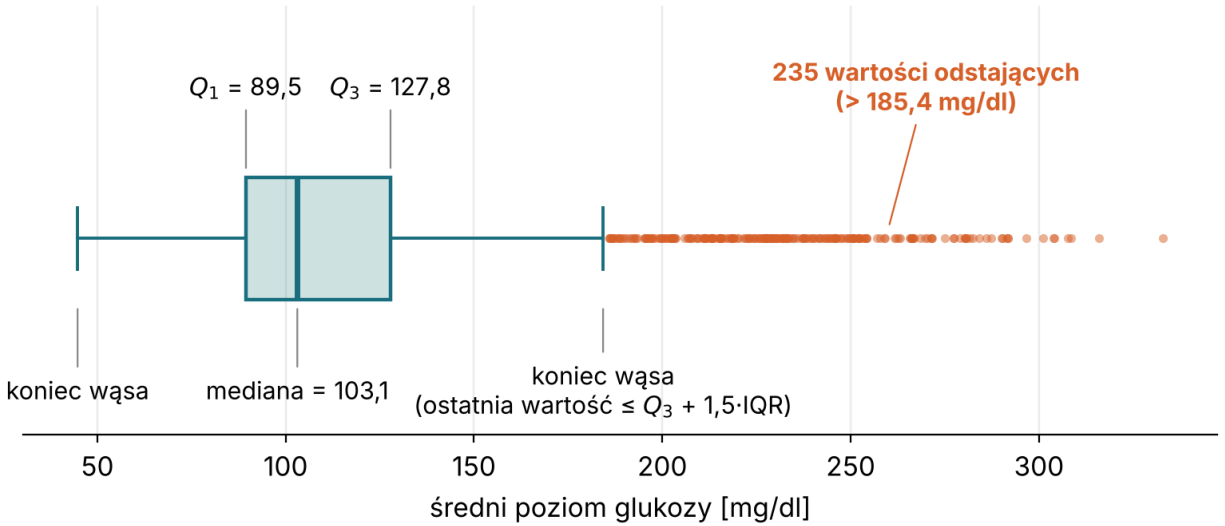
- wykres pudełkowy,
- wykres punktowy.

Zawsze zaczynamy od obejrzenia danych.

Metody liczbowe

- wynik standaryzowany (*z-score*): $z = (x - \overline{x})/s$, np. $|z| > 3$,
- rozstęp międzykwartylowy (IQR): poza $[Q_1 - 1.5 \cdot \mathrm{IQR}, Q_3 + 1.5 \cdot \mathrm{IQR}]$,
- percentyle, np. poniżej 1. lub powyżej 99. percentyla.

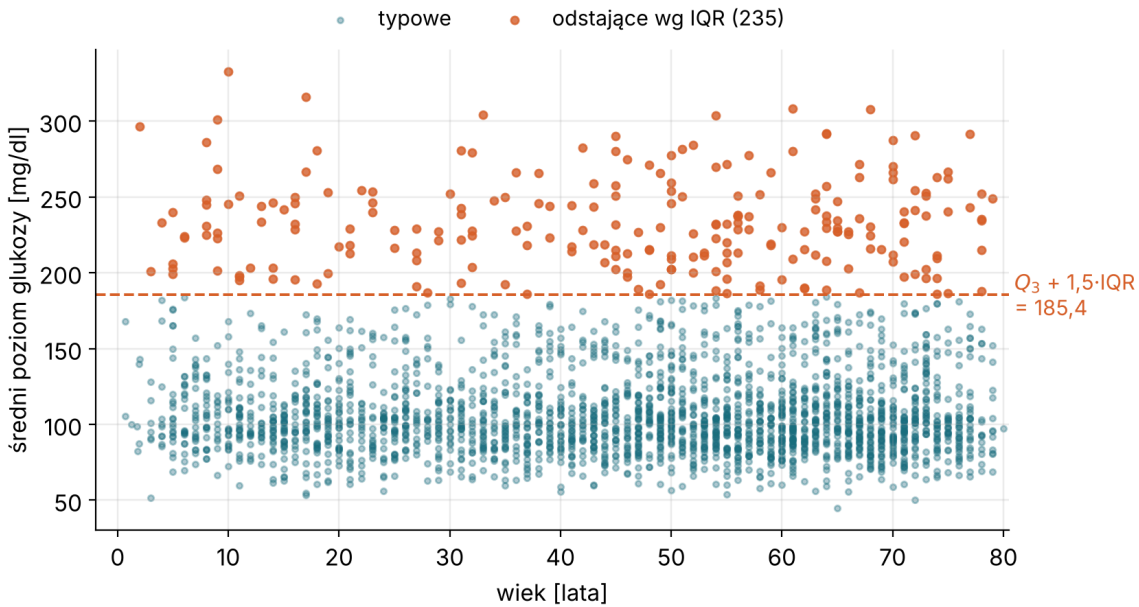
Wykres pudełkowy




```
pacjenci.boxplot(column="avg_glucose_level",
                 vert=False)
```

- Pudełko: od Q_1 do Q_3, kreska — mediana.
- Punkty poza wąsami to **dokładnie** metoda IQR (1,5 · IQR).

Wykres punktowy




```
pacjenci.plot.scatter(x="age", y="avg_glucose_level", s=5)
```

- Pokazuje każdą obserwację, a nie tylko podsumowanie.
- Widać, **gdzie** leżą wartości odstające względem innej cechy.
- Wysoka glukoza występuje w każdym wieku.

Pytanie

10-latek ze średnią glukozą 333 mg/dl — błąd czy rzadki przypadek? Co ten sam poziom oznacza u 70-latka?

Metoda IQR w pandas

```
# po kroku 1: wartości niemożliwe są już NaN
glukoza = pacjenci["avg_glucose_level"]

q1, q3 = glukoza.quantile([0.25, 0.75])
iqr = q3 - q1
dolna, gorna = q1 - 1.5 * iqr, q3 + 1.5 * iqr

odstajace = (glukoza < dolna) | (glukoza > gorna)
odstajace.sum()       # 235  (9% pacjentów!)
```

$Q_1 = 89,5$, $Q_3 = 127,8$, granice: $[31,9; 185,4]$ mg/dl — wszystkie odstające są **powyżej** górnej granicy.

Pytanie

Czy 9% pacjentów to wciąż „wyjątki”? Co to mówi o rozkładzie glukozy?

Metoda z-score w pandas

```
glukoza = pacjenci["avg_glucose_level"]

z = (glukoza - glukoza.mean()) / glukoza.std()

odstajace = z.abs() > 3
odstajace.sum()        # 62
```

$$z = \frac{x - \overline{x}}{s}$$

- z mówi, o ile odchyleń standardowych wartość leży od średniej.
- $\overline{x} = 116,8$, $s = 44,5$ → granica: $116,8 + 3 \cdot 44,5 \approx 250$ mg/dl.
- Dla rozkładu normalnego $|z| > 3$ to ok. 0,3% obserwacji.

To ta sama operacja co **standaryzacja** — wrócimy do niej w części 5.

Pułapka z-score

- $\overline{x}$ i s liczymy **razem** z wartościami odstającymi — one same zawyżają s i „ukrywają się” (**efekt maskowania**).
- W próbie n elementów zawsze $|z| \leq \frac{n-1}{\sqrt{n}}$.
- Dla $n = 10$: $|z| \leq 2,85$ — reguła $|z| > 3$ **nigdy** niczego nie wykryje!

Odporny z-score

$$z_m = 0,6745 \cdot \frac{x - \text{med}}{\text{MAD}}$$

$$\text{MAD} = \text{med}(|x_i - \text{med}|)$$

- Próg: $|z_m| > 3,5$ (Iglewicz, Hoaglin).
- Mediana i MAD nie „uciekają” za wartościami odstającymi.

Metoda percentyli

```
dolna, gorna = glukoza.quantile([0.01, 0.99])
# 61,0 i 271,4 mg/dl

odstajace = (glukoza < dolna) | (glukoza > gorna)
odstajace.sum()     # 54 (27 + 27)
```

Progi	Granice [mg/dl]	Odstające
1. i 99. percentyl	61,0 – 271,4	54
5. i 95. percentyl	72,7 – 225,6	270

- Najprostsza metoda: odcinamy z góry ustalony **procent** skrajnych wartości.
- Nie zakłada żadnego rozkładu.
- Wada: **zawsze** coś znajdzie — także w wieku pacjentów (34 osoby poza 1. i 99. percentylem), choć tam nie ma nic podejrzanego.

Percentyle lepiej służą do **przycinania** (winsoryzacji) niż do wykrywania.

Cztery metody — cztery odpowiedzi

Glukoza (`avg_glucose_level`), 2700 pacjentów:

Metoda	Odstające
$\lvert z\rvert > 3$	62
IQR (1,5)	235
odporny $\lvert z_m\rvert > 3,5$	223
percentyle 1–99	54

Skośność glukozy: **1,8**; wieku: −0,3 — dla wieku żadna metoda nic nie wykrywa.

- Reguły „3 sigma” i IQR zakładają rozkład **mniej więcej symetryczny**.
- Glukoza, ceny, dochody, czasy mają zwykle **długi prawy ogon** — wtedy „odstających” jest dużo.

Najpierw **obejrzyj rozkład** (histogram), dopiero potem wybierz metodę i próg. Często pomaga logarytm.

Krok 3: nielogiczne kombinacje kategorii

```
# tabela krzyżowa pokazuje „dziwne” pary
pd.crosstab(pacjenci["work_type"],
            pacjenci["smoking_status"])

dziecko_pali = (
    (pacjenci["work_type"] == "children")
    & pacjenci["smoking_status"]
        .isin(["smokes", "formerly smoked"]))

maluch_student = (
    (pacjenci["age"] <= 3)
    & (pacjenci["work_type"] == "student"))

dziecko_pali.sum(), maluch_student.sum()  # (15, 1)
```

- Każda wartość **osobno** jest poprawna: „dziecko”, „pali”.
- Niemożliwa jest dopiero ich **kombinacja**.
- Takich błędów nie wykryje ani IQR, ani z-score — potrzebne są **reguły** z wiedzy dziedzinowej.

Krok 4: odstające w wielu wymiarach

- Obserwacja może być typowa w **każdej** zmiennej osobno, a nietypowa w ich **kombinacji**.
- Przykład: 3 lata doświadczenia — typowe; 12 tys. zł pensji — typowe; razem — podejrzane.
- Pomysł: mierzymy **odległość** punktu od środka chmury danych.

Pracownik	Doświadczenie	Pensja
Celina	3 lata	5,5 tys.
Iwona	15 lat	10,0 tys.
Jakub	**3 lata**	**12,0 tys.**
Lena	20 lat	8,0 tys.

Fragment danych z notatnika (12 pracowników).

Odległość od środka: euklidesowa

$$d^2 = \sum_j \left(\frac{x_j - \overline{x}_j}{s_j} \right)^2$$

- „Twierdzenie Pitagorasa” w p wymiarach — suma kwadratów z-score’ów.
- **Standaryzacja jest konieczna:** glukoza ma odch. std. ok. 45, BMI ok. 6 — bez niej liczyłaby się prawie tylko glukoza.
- Dla danych normalnych d^2 ma rozkład χ^2 z p stopniami swobody → próg z kwantyla.

```
from scipy.stats import chi2

X = pacjenci[["age", "bmi",
              "avg_glucose_level"]].dropna()

Z = (X - X.mean()) / X.std()   # standaryzacja
d2 = (Z ** 2).sum(axis=1)

prog = chi2.ppf(0.99, df=X.shape[1])  # 11,34
(d2 > prog).sum()      # 73 z 2440 pacjentów
```

Krok 1 najpierw — wartości niemożliwe zawyżyłyby średnie i odchylenia.

Najpierw obejrzyj rozkłady

- Glukoza ma **długi prawy ogon**: mediana 103 mg/dl, maksimum 333.
- Z 73 „odstających” aż **54** trafia tam głównie przez glukozę.
- Odległość wskazuje wtedy po prostu ogon jednej cechy.

	glukoza	log(glukoza)
skośność	1,80	0,91
odstający ($d^2 > 11,34$)	73	**31**

```
X["avg_glucose_level"].skew()      # 1.80

X_log = X.assign(avg_glucose_level=
    np.log(X["avg_glucose_level"]))

Z = (X_log - X_log.mean()) / X_log.std()
d2_log = (Z ** 2).sum(axis=1)
(d2_log > prog).sum()              # 31
```

Próg χ^2 zakłada rozkład zbliżony do normalnego — dla skośnych cech najpierw logarytm.

Gdy cechy są skorelowane: odległość Mahalanobisa

$$D^2 = (\boldsymbol{x} - \boldsymbol{\mu})^T \boldsymbol{S}^{-1} (\boldsymbol{x} - \boldsymbol{\mu})$$

- $\boldsymbol{S}$ — macierz kowariancji.
- Euklides rysuje wokół środka **okrąg**, Mahalanobis **elipsę** ułożoną wzdłuż trendu.
- Dla cech nieskorelowanych daje to samo co euklidesowa — u pacjentów ($r \approx 0,02$) **wystarcza euklidesowa**.

Doświadczenie a pensja (korelacja 0,43):

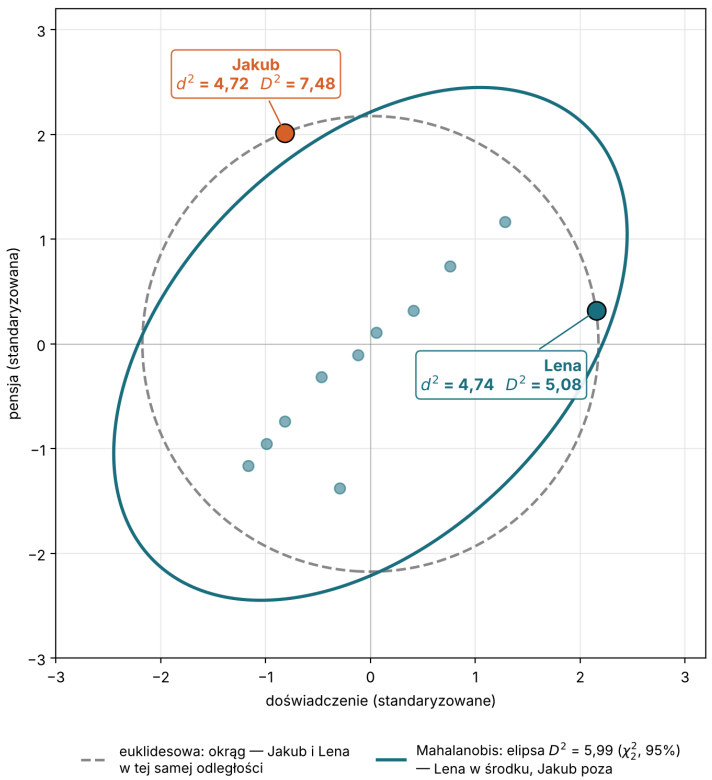
	d^2	D^2
Lena	4,74	5,08
Jakub	4,72	**7,48**

Euklides: obaj tak samo daleko. Mahalanobis: Jakub idzie **pod prąd trendu**.

Gdy cechy są skorelowane: odległość Mahalanobisa

$$D^2 = (\boldsymbol{x} - \boldsymbol{\mu})^T \boldsymbol{S}^{-1} (\boldsymbol{x} - \boldsymbol{\mu})$$

- S — macierz kowariancji.
- Euklides rysuje wokół środka **okrąg**, Mahalanobis **elipsę** ułożoną wzdłuż trendu.
- Doświadczenie a pensja (korelacja 0,43): Jakub i Lena są równie daleko po euklidesowemu, ale tylko Jakub idzie **pod prąd trendu**.
- Dla cech nieskorelowanych obie miary dają to samo — u pacjentów ($r \approx 0,02$) **wystarcza euklidesowa**.



Dla chętnych: Isolation Forest i LOF

- Gdy chmura danych nie jest „elipsą” (kilka skupisk, nieliniowe zależności).
- **Isolation Forest** — odstające łatwo „odizolować” losowymi podziałami.
- **LOF** (*Local Outlier Factor*) — punkt w dużo rzadszym otoczeniu niż jego sąsiedzi.
- LOF działa na odległościach — cechy trzeba **wystandaryzować**.

```
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.preprocessing import StandardScaler

iso = IsolationForest(contamination=0.02,
                      random_state=0)
etykiety = iso.fit_predict(X)   # -1 = odstająca

X_std = StandardScaler().fit_transform(X)
etykiety = LocalOutlierFactor(
    n_neighbors=20).fit_predict(X_std)
```

Od najprostszej metody do najbardziej złożonej

Krok	Metoda	Co wykrywa? (pacjenci)
1	progi z wiedzy dziedzinowej	wartości niemożliwe: 300 BMI, 300 glukozy
2	IQR, z-score, MAD — jedna zmienna	wartości skrajne w jednej cesze
3	reguły logiczne dla kategorii	15 palących dzieci, 1 maluch-student
4	odległość od środka (euklidesowa, po log)	nietypowe **kombinacje** liczb: 31 pacjentów
5	Isolation Forest, LOF	złożone kształty danych

Co zrobić z wartością odstającą?

Strategia	Kiedy?
poprawić / zamienić na NaN	ewidentny błąd (literówka, kod `-999`, zła jednostka)
usunąć wiersz	błąd, którego nie da się poprawić — **udokumentuj**, ile usunięto
przyciąć (winsoryzacja)	prawdziwe, ale ekstremalne wartości psują model
przekształcić (`log`)	rozkład prawoskośny: ceny, dochody, czasy
metody odporne	mediana zamiast średniej, regresja Hubera, drzewa decyzyjne
oznaczyć flagą	wartość może nieść informację (np. podejrzana transakcja)

Przycinanie i transformacja w pandas

```
# Winsoryzacja: wartości poza 1. i 99. percentylem → do granicy
dolna, gorna = titanic["Fare"].quantile([0.01, 0.99])
titanic["Fare_clip"] = titanic["Fare"].clip(dolna, gorna)
# max: 512,33 → 249,01

# Logarytm (log1p = log(1 + x) — działa też dla zera)
titanic["Fare_log"] = np.log1p(titanic["Fare"])

# Flaga zamiast usuwania
titanic["Fare_odstaje"] = titanic["Fare"] > gorna
```

Uwaga

Progi (percentyle, IQR) wyznaczamy na **zbiorze treningowym** i stosujemy do testowego — inaczej informacja z testu „przecieka” do modelu.

5

Jak radzić sobie z brakującymi wartościami?

Brakujące wartości

Pola, w których nie ma danych.

Przyczyny:

- błąd przy wprowadzaniu danych,
- ochrona prywatności,
- brak odpowiedzi w ankiecie,
- problemy techniczne przy zbieraniu danych.

Jeden z najczęstszych problemów i kluczowy etap przetwarzania wstępnego.

```
# Czy są braki? (isna == isnull)
titanic.isna()

# Liczba braków w kolumnach
titanic.isna().sum()

# Procent braków
titanic.isna().mean() * 100

# Łączna liczba braków
titanic.isna().sum().sum()
```

Czym jest NaN i dlaczego jest wyjątkowy?

- **NaN** (*Not a Number*) — specjalna wartość oznaczająca brak danych.
- `NaN` nie jest równy niczemu — nawet sobie. Dlatego do wykrywania braków służą funkcje `isna()`.
- `0` i pusty napis `""` to dla pandas **zwykłe wartości**, a nie braki.

```
np.nan == np.nan      # False (!)
pd.isna(np.nan)       # True

df = pd.DataFrame({
    "A": [1, np.nan, 3],    # NaN  — brak
    "B": [4, None, 6],      # None — też brak
    "C": [7, 0, 9],         # 0    — NIE brak!
    "D": ["x", "", "z"],    # ""   — NIE brak!
})
df.isna().sum()     # A: 1, B: 1, C: 0, D: 0
```

Braki „w przebraniu”

`isna()` widzi tylko `NaN`/`None`. Braki bywają zapisane jako tekst (`"NA"`, `"brak"`, `"?"`, `"-"`) lub liczba (`0`, `-1`, `-999`):

```
# Podejrzane wartości w kolumnie liczbowej
(titanic["Fare"] == 0).sum()      # 15

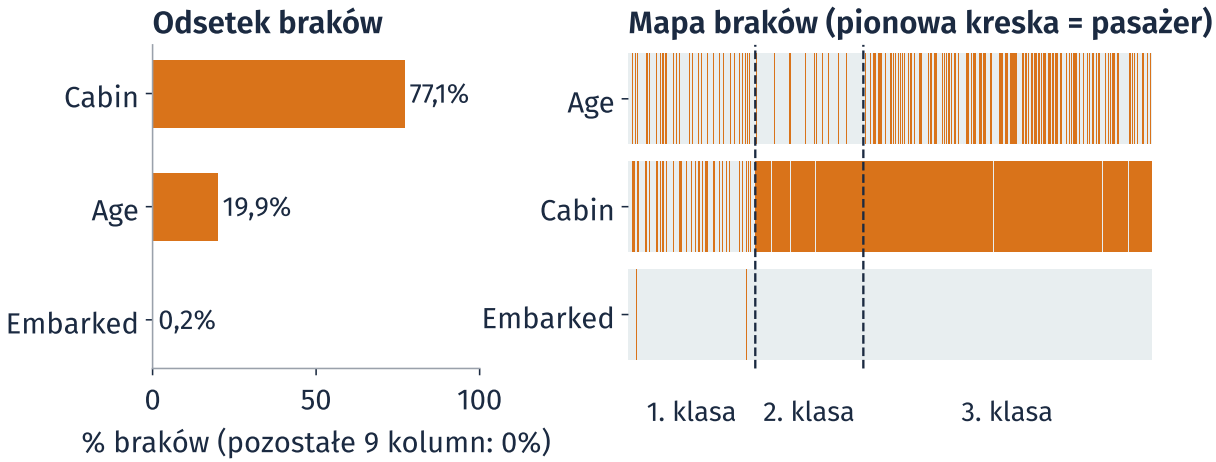
# Napisy udające brak
df["kolumna"].value_counts()       # "?", "-", "brak"...

# Przy wczytywaniu…
pd.read_csv("plik.csv", na_values=["brak", "?", -999])
# …albo po wczytaniu
df = df.replace(["?", "-", -999], np.nan)
```

Pytanie

15 pasażerów Titanica ma bilet za 0 £. To brak danych, błąd czy prawda (np. bilet służbowy)? Jak to sprawdzić?

Gdzie są braki?




```python
braki = titanic.isna()
# odsetek braków
braki.mean().sort_values().plot.barh()
# mapa braków (wiersze posortowane wg klasy)
kolejnosc = titanic.sort_values("Pclass").index
sns.heatmap(braki.loc[kolejnosc], cbar=False)
```

Braki tworzą **wzór**: kabiny brakuje niemal wyłącznie w 2. i 3. klasie. Mapę braków warto posortować według zmiennej, od której braki mogą zależeć.

Dlaczego brakuje? Trzy mechanizmy

Mechanizm	Znaczenie	Przykład
MCAR całkowicie losowe	brak nie zależy od niczego	czujnik losowo gubi pomiary; kartka ankiety zalana kawą
MAR losowe warunkowo	brak zależy od **innych**, obserwowanych zmiennych	w 3. klasie Titanica rzadziej zapisywano wiek
MNAR nielosowe	brak zależy od **samej** brakującej wartości	osoby o wysokich dochodach nie podają dochodu

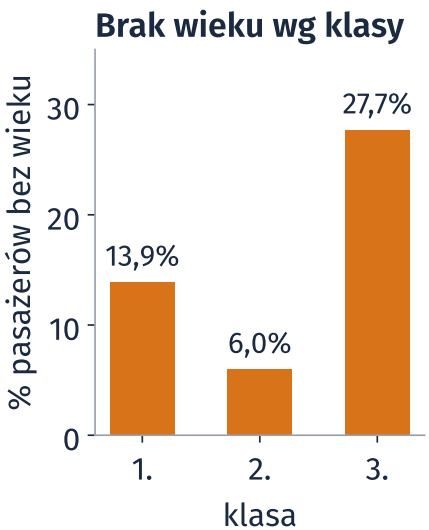
Tylko przy MCAR usunięcie wierszy nie zniekształca wyników. Przy MAR pomagają metody wykorzystujące inne zmienne; MNAR wymaga wiedzy dziedzinowej.

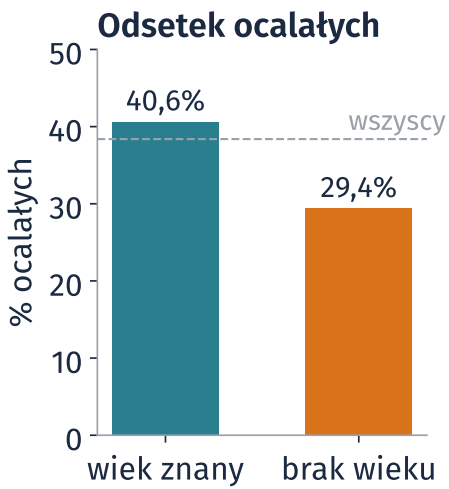
Czy braki wieku na Titanicu są losowe?

```python
brak_wieku = titanic["Age"].isna()

# odsetek braków w każdej klasie
brak_wieku.groupby(
    titanic["Pclass"]).mean()

# odsetek ocalałych
titanic.groupby(brak_wieku)[
    "Survived"].mean()
```





Braki **nie** są losowe (to nie MCAR): w 3. klasie wieku brakuje 4,6 razy częściej niż w 2., a osoby bez wieku rzadziej przeżyły. Usunięcie tych 177 osób **zawyży** odsetek ocalałych.

Jak radzić sobie z brakami?

- **Usunąć** wiersze (lub kolumny) z brakami.
- **Uzupełnić ręcznie**, jeśli znamy prawdziwą wartość.
- **Uzupełnić miarą tendencji centralnej:**
  - **średnią** — cechy liczbowe o w miarę symetrycznym rozkładzie,
  - **medianą** — cechy liczbowe skośne (np. dochód) i cechy porządkowe,
  - **dominantą** (modą) — cechy kategoryczne.
- **Oznaczyć brak** dodatkową kolumną, np. `Age_brak = True` — sam fakt braku bywa informacją.

Uwaga

Najpierw sprawdź, **co** oznacza brak. W zbiorze Ames `NA` w kolumnie `PoolQC` znaczy „brak basenu” — tego nie uzupełnia się średnią.

Usuwanie brakujących wartości: `dropna()`

```
# Usuń wiersze z DOWOLNĄ brakującą wartością (domyślnie)
titanic.dropna()                        # zostaje 183 z 891 wierszy

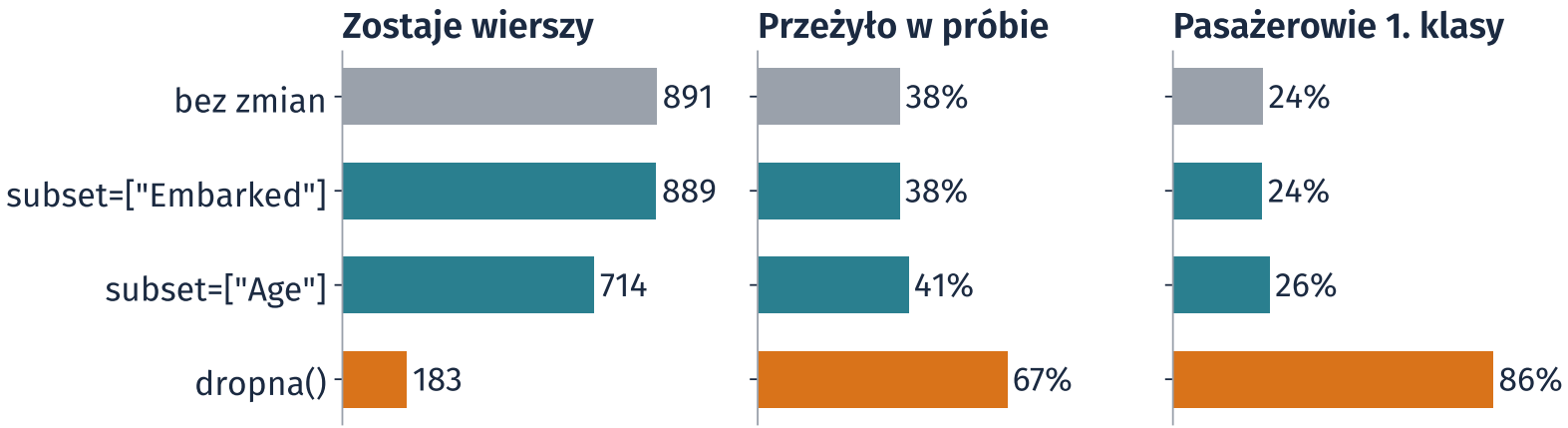
# Usuń wiersze, w których brakuje WSZYSTKICH wartości
titanic.dropna(how="all")               # 891 — takich wierszy nie ma

# Usuń wiersze z brakami tylko w wybranych kolumnach
titanic.dropna(subset=["Age"])          # 714

# Usuń KOLUMNY z brakami
titanic.dropna(axis=1)                  # znikają Age, Cabin i Embarked

# Najpierw usuń „dziurawą” kolumnę, potem wiersze
titanic.drop(columns="Cabin").dropna()  # 712
```

Ile kosztuje usuwanie?



Po `dropna()` zostaje 183 pasażerów: **67%** z nich przeżyło, a **86%** podróżowało 1. klasą (w całym zbiorze: 38% i 24%). To już nie jest próbka pasażerów Titanica.

Usuwaj **kolumnę**, gdy braków jest bardzo dużo i nie niosą informacji; **wiersze** — gdy braków jest mało i są losowe.

Uzupełnianie braków: `fillna()`

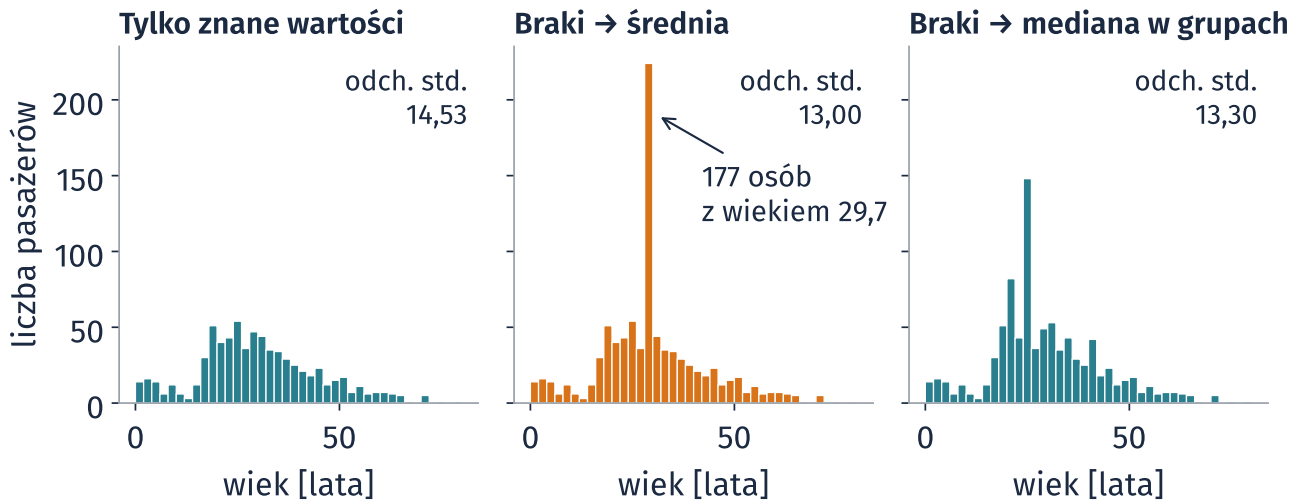
```
# Wiek — średnia (najprostsza metoda; za chwilę zobaczymy jej koszt)
wiek_srednia = titanic["Age"].fillna(titanic["Age"].mean())    # 29,7

# Gdyby brakowało ceny biletu (rozkład skośny) — mediana
titanic["Fare"].fillna(titanic["Fare"].median())

# Port zaokrętowania (cecha kategoryczna) — dominanta
titanic["Embarked"] = titanic["Embarked"].fillna(
    titanic["Embarked"].mode()[0])                              # "S"
```

Przypisujemy wynik z powrotem do kolumny — w pandas 3.0 nie używamy `inplace=True` na wybranej kolumnie.

Cena prostego uzupełniania



- Po uzupełnieniu średnią **177 pasażerów (20%)** ma dokładnie 29,7 roku — na histogramie powstaje sztuczny „słup”.
- Zmienność jest zaniżona, a korelacje osłabione (wiek–klasa: z −0,37 do −0,33).

> Zawsze porównaj rozkład **przed** i **po** uzupełnieniu.

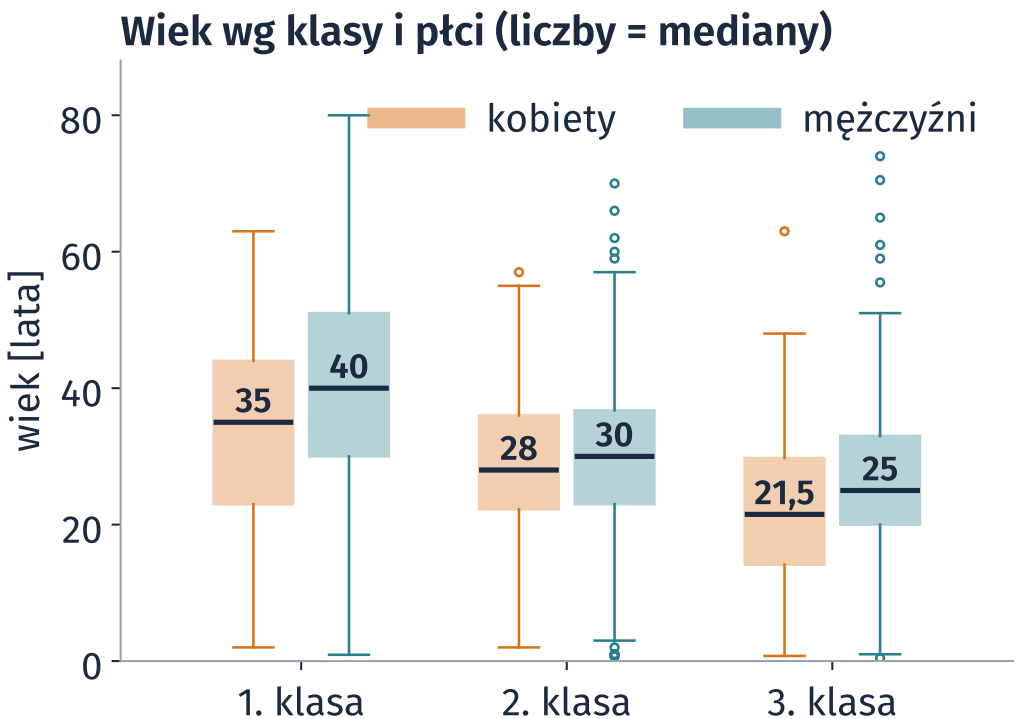
Uzupełnianie w grupach

```python
# flaga braku — przed uzupełnieniem!
titanic["Age_brak"] = titanic["Age"].isna()

mediana_grupy = (titanic
    .groupby(["Pclass", "Sex"])["Age"]
    .transform("median"))

titanic["Age"] = titanic["Age"].fillna(
    mediana_grupy)
```

`transform` zwraca wynik o długości ramki — każdemu wierszowi medianę **jego** grupy.



Zaawansowane uzupełnianie braków

- Uzupełnianie **najbardziej prawdopodobną** wartością przewidzianą przez model:
  - regresja,
  - drzewa decyzyjne,
  - algorytm k najbliższych sąsiadów (KNN).
- Uzupełnianie średnią lub medianą **spłaszcza rozkład** i zaniża zmienność — modele uwzględniają zależności między cechami.

Titanic — nasze decyzje

Kolumna	Braki	Propozycja
`Cabin`	77,1%	nie uzupełniamy numeru kabiny — tworzymy cechę `ma_kabine`: przeżyło ok. 67% osób ze znaną kabiną i 30% bez niej
`Age`	19,9%	mediana w grupach (klasa × płeć) + flaga `Age_brak`; ewentualnie `IterativeImputer`
`Embarked`	2 wiersze	dominanta (`"S"`) albo usunięcie 2 wierszy

Pytanie

Czy dla `Cabin` brak to MCAR, MAR czy MNAR? (Wskazówka: mapa braków na początku podrozdziału.)

Ściągawka: brakujące wartości

Diagnoza

1. Policz braki (`isna().sum()`).
2. Znajdź braki „w przebraniu”.
3. Narysuj mapę braków.
4. Sprawdź, czy braki zależą od innych cech.

Decyzja

1. Mało i losowo → usuń wiersze.
2. Bardzo dużo → usuń kolumnę lub zrób flagę.
3. Inaczej → uzupełnij (grupy, model) + flaga.
4. Porównaj rozkład przed i po.

Każdą decyzję **zapisz** — ile wierszy usunięto, czym uzupełniono i dlaczego.

Źródła

- W. McKinney, *Python w analizie danych*, Helion.
- A. J. Gutman, J. Goldmeier, *Analityk danych. Przewodnik po data science, statystyce i uczeniu maszynowym*, Helion.
- B. Iglewicz, D. C. Hoaglin, *How to Detect and Handle Outliers*, ASQC Quality Press, 1993.
- P. C. Mahalanobis, *On the generalised distance in statistics*, Proc. National Institute of Sciences of India 2 (1936) 49–55.
- Dokumentacja pandas: *IO tools* — pandas.pydata.org/docs.
- Dokumentacja scikit-learn: *Novelty and outlier detection* — scikit-learn.org.
- deepsense.ai, *EDA Benchmark* (2026), CC BY-NC-ND 4.0 — github.com/deepsense-ai/eda-benchmark.
- dane.gov.pl, otwarte.miasto.lodz.pl, ourworldindata.org, huggingface.co/datasets, archive.ics.uci.edu, kaggle.com.
- Liczby dla Titanica i danych o pacjentach policzone w pandas 3.0 i scikit-learn 1.9.